

Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Classes e Objetos

... os principais atores em um programa Java são os *objetos*.

todo objeto é uma instância de uma **classe**

atributos armazenam dados

fornece os **métodos** para acessar e modificar dados

Classes e Objetos

```
modificador class NomeDaClass {  
    modificador tipo nomeAtributo1;  
    modificador tipo nomeAtributo2;  
    . . .  
  
    public NomeDaClasse() { ... }  
  
    modificador tipo nomeMetodo1(){ return algo; }  
  
    modificador void nomeMetodo2(){ fazAlgo; }  
    . . .  
}
```

Classes e Objetos

```
modificador class NomeDaClass {  
    modificador tipo nomeAtributo1;  
    modificador tipo nomeAtributo2;  
    . . .  
  
    public NomeDaClasse() { ... }  
  
    modificador tipo nomeMetodo1(){ return algo; }  
    modificador void nomeMetodo2(){ fazAlgo; }  
    . . .  
}
```

variáveis de instância

construtor

métodos

Classes e Objetos

```
public class Counter {  
    private int count;  
  
    public Counter() { count = 0; }  
  
    public int getCount() {  
        return count;  
    }  
  
    public void increment() { count++; }  
    public void decrement() { count--; }  
}
```

Classes e Objetos

```
class Main {  
    public static void main(String[] args) {  
        Counter c, d;  
        c = new Counter();  
        d = new Counter();  
  
        c.increment();  
        d.decrement();  
  
        System.out.println(c.getCount());  
        System.out.println(d.getCount());  
    }  
}
```

Modificadores de atributos

De visibilidade:

public: podem ser acessados fora da classe

protected: apenas métodos do mesmo pacote (diretório) ou subclasse podem acessar

private: acessado apenas dentro da própria classe

nenhum: *amigável*, que significa que podem ser acessadas dentro do mesmo pacote (diretório).

De uso (inseridos após um modificador de visibilidade):

static: define um atributo de classe, isto é, existem mesmo sem a criação de um objeto

final: define um atributo que deve ser inicializado e não modificado

Modificadores de atributos

```
class Counter {  
    private int count;  
    public static int MIN;  
    public static final int MAX = 10;  
  
    public Counter() { count = 0; }  
  
    public int getCount() {  
        return count;  
    }  
  
    public void increment(){ if (count < MAX) count++; }  
    public void decrement(){ if (count > MIN) count--; }  
}
```


Modificadores de atributos

```
class Main {  
    public static void main(String[] args) {  
        Counter c, d;  
        c = new Counter();  
        d = new Counter();  
  
        c.increment();  
        d.decrement();  
  
        Counter.MIN = -5;  
  
        System.out.println(c.MIN);  
        System.out.println(d.MIN);  
    }  
}
```

Modificadores de métodos

Visibilidade:

public: podem ser acessados fora da classe

protected: apenas métodos do mesmo pacote (diretório) ou subclasse podem chamar um método protegido

private: acessado apenas dentro da própria classe

nenhum: *amigável*, que significa que podem ser acessados por objetos de classes do mesmo pacote (diretório).

Uso:

static: define um método de classe, isto é, existem mesmo sem a criação de um objeto e podem modificar atributos **static**

final: define um método que não pode ser sobrescrito por subclasses

Modificadores de métodos

```
class Counter {  
    ...  
    private static int MIN;  
    ...  
  
    ...  
    public static void setMIN(int v) {  
        MIN = v;  
    }  
  
    public static int getMIN() {  
        return MIN;  
    }  
}
```

Modificadores de métodos

```
class Main {  
    public static void main(String[] args) {  
        Counter c, d;  
        c = new Counter();  
        d = new Counter();  
  
        c.increment();  
        d.decrement();  
  
        Counter.setMIN(-5);  
  
        System.out.println(c.MIN);  
        System.out.println(Counter.getMIN());  
    }  
}
```

Exercício

1. Escreva uma classe para representar um Cartão de Crédito: CartaoCredito.

Possuem atributos:

- número
- nome do titular
- nome do banco
- saldo
- limite

Métodos para manipular o uso do cartão e um método static para imprimir os dados de um cartão passado como parâmetro.

2. Escreva um programa de teste contendo uma carteira com 3 cartões. Execute operações aleatórias na carteira e, ao final, imprima o balanço geral.

Sobrecarga de métodos

Assinatura de um método

Sintaxe: `nomeDoMetodo (tipo, tipo, tipo, ...)`

Em Java, uma classe pode conter métodos com diferentes assinaturas. Isso significa que, apenas variando o tipo e a quantidade de parâmetros, os métodos podem ter o mesmo nome.

```
public class DataArtist {  
    public void draw(String s) { ... }  
    public void draw(int i) { ... }  
    public void draw(double f) { ... }  
    public void draw(int i, double f) { ... }  
}
```

Passagem de parâmetros

O valor de um tipo primitivo é copiado...

A referência a um objetivo é copiada...

```
class Main {  
    public static void altCounter(Counter c){  
        c.setCount(10);  
    }  
    ...  
    altCounter(c);  
    ...  
}
```

Exercício

1. Crie uma classe para representar um ponto em um plano bidimensional (x,y).

- crie atributos e métodos para a manipulação dos atributos
- Crie uma classe para representar um retângulo, possuindo um ponto de origem (objeto da classe ponto), uma altura e uma largura.

crie métodos para manipular apropriadamente os atributos, incluindo a modificação da origem (mover o retângulo) e para calcular a sua área

crie um construtor para um retângulo em um ponto específico, altura e largura

crie um construtor para um retângulo na origem (0,0)

DESAFIO: Como alterar a classe Retangulo para informar quantos objetos foram instanciados quando se desejar saber?

Empacotamento

Uso da palavra reservada **package** no início do arquivo:

```
package nome_pacote;
```

```
....
```

<https://docs.oracle.com/javase/tutorial/java/package/namingpkgs.html>